

Using Evolution Strategies to Solve Sparse-Reward Tasks

Evaluating Gradient-Free Optimization in Reinforcement Learning

Contents

1	Introduction	1
2	Background: Evolution Strategies	2
3	Task: Wordle as a Sparse-Reward RL Problem	3
4	Stage 1: Gridworld Proof of Concept	3
5	Stage 2: Initial Wordle Experiments with Gemma	3
6	Stage 3: Curriculum Learning and Optimization Attempts	4
7	REINFORCE Baseline	5
8	Stage 4: Diagnosing the ES Failure	5
9	Stage 5: Pivot to Qwen and Character-Level Wordle	6
10	Stage 6: Prime Intellect Checkpoint Experiments	6
10.1	SFT Initialization	7
10.2	RL Initialization	7
10.3	RL Initialization with Normalized ES Updates	8
11	Overall Results	8
12	Discussion	9
13	Limitations and Caveats	10
14	Conclusion	11

1 Introduction

Evolution Strategies (ES) are an appealing alternative to standard gradient-based reinforcement learning methods because they optimize policies through parameter-space perturbations rather

than backpropagating through sampled trajectories. This makes ES especially attractive in settings where rewards are sparse, gradients are noisy, or distributed communication costs dominate training. In principle, ES only requires scalar rewards from policy rollouts, making it simple and scalable.

In this project, we investigated whether ES could solve sparse-reward language-agent tasks, using Wordle as the main benchmark. Wordle is a useful testbed because it is simple to specify but difficult for reinforcement learning: the agent receives limited feedback, must reason over multiple turns, and only receives full success reward when it guesses the hidden word within six attempts. We began with a smaller Gridworld environment to validate our ES implementation, then progressively moved toward more realistic Wordle experiments with pretrained language models, curriculum learning, supervised warm starts, LoRA-based parameter-efficient tuning, character-level actions, and stronger external checkpoints.

The project ultimately became a study of where ES fails. ES succeeded on Gridworld, showing that the implementation could solve a small sparse-reward control task. However, when scaled to Wordle, ES repeatedly failed to produce sustained improvement despite many optimization changes. We first tried Gemma-based Wordle policies with trainable heads and LoRA adapters, then introduced supervised warm starts and curriculum learning. When these methods still failed, we switched to Qwen-based Wordle models and a character-by-character action formulation to reduce the action-space difficulty. Finally, we tested ES starting from Prime Intellect’s released Wordle checkpoints, including both their SFT checkpoint and their stronger RL checkpoint. Even from these stronger initializations, ES did not produce reliable Wordle improvement.

Our main result is therefore negative but informative: ES can solve small sparse-reward environments, but in Wordle-scale language-agent settings, the reward signal remained too brittle and sparse for parameter-space search to make consistent progress. Stronger initialization and stabilized ES updates improved the geometry of training, but not task-level performance.

2 Background: Evolution Strategies

Evolution Strategies optimize a policy by directly perturbing model parameters and using the resulting rewards to estimate an update direction. Given parameters θ , ES samples Gaussian perturbations ϵ_i and evaluates perturbed policies of the form

$$\theta'_i = \theta + \sigma \epsilon_i.$$

Each perturbed policy is evaluated in the environment, producing a scalar fitness value F_i . These fitness values are then used to estimate a gradient-like update direction:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N\sigma} \sum_{i=1}^N F_i \epsilon_i.$$

Unlike policy-gradient methods such as REINFORCE, ES does not require differentiating through action probabilities or storing full trajectories for backpropagation. Workers only need to return scalar rewards and perturbation seeds, so the communication cost can be much lower than standard distributed gradient-based RL.

This property makes ES especially interesting for sparse-reward tasks. If useful reward differences can be detected across perturbations, ES can search directly in parameter space without relying on dense token-level or action-level gradients. However, this same property becomes a weakness when almost all perturbations receive identical or near-identical reward. In that case, the

ES gradient estimate becomes dominated by noise, and the method has little signal about which parameter directions are useful.

3 Task: Wordle as a Sparse-Reward RL Problem

We used Wordle as the main sparse-reward language-agent benchmark. The goal is to guess a hidden five-letter word in at most six turns. After each guess, the environment returns feedback for each character: green if the letter is correct and in the correct position, yellow if the letter appears in the word but in a different position, and gray otherwise.

The natural sparse reward is binary:

$$R = \begin{cases} 1 & \text{if the model guesses the hidden word within six turns,} \\ 0 & \text{otherwise.} \end{cases}$$

This makes Wordle difficult for RL because most early policies rarely solve the game. Without a warm start, many rollouts receive zero reward, so both policy-gradient methods and ES struggle to identify useful updates.

To make the task somewhat less sparse, we also used shaped rewards based on partial Wordle feedback. For example, green letters received higher reward than yellow letters, and faster solves received additional bonus reward. However, even with shaping, the reward landscape remained difficult: many policies could receive partial feedback without learning a robust multi-turn solving strategy.

4 Stage 1: Gridworld Proof of Concept

Before applying ES to Wordle, we first tested it on a small Gridworld environment. The environment was an 8×8 grid with one goal state and several obstacles. The agent received positive reward for reaching the goal and small negative reward for hitting obstacles. The policy was a two-layer MLP with approximately 8,580 trainable parameters.

This experiment validated the basic ES implementation. ES successfully reached 100% success on the base Gridworld layout. This showed that the optimization loop, perturbation sampling, reward evaluation, and parameter updates were functioning correctly on a small sparse-reward task.

We then explored curriculum learning in Gridworld by gradually perturbing the layout and continuing training across related environments. The purpose was to test whether ES could learn policies that generalized beyond a single fixed environment. These experiments motivated applying a similar curriculum idea to Wordle, where the vocabulary size could be gradually increased.

5 Stage 2: Initial Wordle Experiments with Gemma

After Gridworld, we moved to Wordle using a pretrained Gemma 1B model as the base policy. The initial action space treated each full five-letter Wordle guess as a single action. This meant the policy had to choose among a vocabulary of possible words, rather than generating the guess one character at a time.

Our early setup used a frozen Gemma backbone with a trainable policy head. The head mapped the model’s hidden state to logits over possible Wordle guesses. We also experimented with LoRA

adapters on the language model, allowing a small number of low-rank trainable parameters while keeping the full pretrained model mostly frozen.

A key issue was that a randomly initialized Wordle policy almost never won. This made the ES reward signal nearly zero everywhere. To address this, we introduced a supervised warm start. The warm start generated Wordle-like training examples by choosing a secret word, simulating several random consistent guesses, and training the model to predict the secret from the accumulated clues. The goal was to place the policy near a region of parameter space where it could sometimes solve the task, giving ES a nonzero signal to refine.

The warm start improved initial Wordle behavior, but ES still did not produce sustained learning. In many runs, performance either stayed flat after the warm start or quickly returned to the noise floor. This suggested that the warm start helped the model memorize or imitate local patterns, but did not create a robust reward landscape where ES could reliably improve the policy.

6 Stage 3: Curriculum Learning and Optimization Attempts

We then tried to make the Wordle task easier through curriculum learning. Instead of immediately training on the full Wordle vocabulary, we restricted the secret-word set to smaller vocabularies and gradually expanded it. A typical schedule used vocabulary sizes such as

16, 32, 64, 128, 256, 512, 1024.

The hope was that ES could first learn a reliable strategy on a small word set, then retain and generalize that strategy as the vocabulary became larger. We also varied the LoRA rank, trying ranks from very small values up to much larger adapters. The motivation was to trade off trainability against ES variance: larger trainable parameter spaces are more expressive, but they also make parameter-space search noisier.

In addition to curriculum learning and LoRA rank changes, we tried several ES stabilization and variance-reduction techniques:

- stronger and weaker supervised warm starts,
- different curriculum schedules,
- different LoRA ranks,
- exponential moving average momentum,
- antithetic sampling using paired $+\epsilon$ and $-\epsilon$ perturbations,
- longer training horizons,
- reward shaping based on partial Wordle feedback.

Despite these changes, the qualitative result remained the same. ES did not show sustained improvement on Wordle. The model sometimes produced isolated evaluation spikes, but these did not persist across checkpoints and did not become a stable solving policy.

7 REINFORCE Baseline

We also implemented a REINFORCE baseline so we could compare ES against a standard on-policy policy-gradient method on the same task, the same backbone, and the same warm-start budget. REINFORCE updates the policy parameters using sampled trajectories and the log-probabilities of the actions that produced reward:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right].$$

Unlike ES, REINFORCE differentiates through the policy’s action distribution, which gives it a much more direct connection between selected actions and reward. The estimator can still have high variance under sparse rewards, but it does not pay the dimensionality cost that ES pays when the trainable parameter space is large.

Our REINFORCE setup matched the matched-comparison ES setup as closely as practical. We used the Gemma 3 1B IT backbone with a small trainable linear policy head, a restricted 32-word action space, 40 training iterations of 16 episodes each, and a 400-step supervised warm start generated by the same procedure used by the ES runs (random feedback-consistent guesses followed by cross-entropy toward the hidden secret). Greedy evaluation was run after training to report deterministic task-level performance.

Configuration	Success rate	Mean turns
REINFORCE, with warm start	16.7%	5.50
REINFORCE, without warm start	41.7%	5.00

Table 1: Greedy evaluation of the REINFORCE baseline on a 32-word restricted Wordle action space with Gemma 3 1B IT. The without-warm-start configuration outperformed the warm-started configuration in this run, which we read as evidence that the warm-start procedure pulled the policy into a local region that REINFORCE’s gradient signal then had trouble leaving — not as a general claim that warm starts are harmful.

Two observations are relevant for the comparison with ES. First, neither REINFORCE configuration was heavily tuned, and the without-warm-start run unexpectedly produced the stronger headline number; we therefore treat both numbers as rough baselines rather than as a precise ranking of warm-start vs. no-warm-start under REINFORCE. Second, both REINFORCE configurations clearly outperformed the corresponding ES runs on the matched setup: the ES path on the full-vocabulary curriculum did not produce stable evaluation success, and even the Week 16 Qwen runs from Prime Intellect’s checkpoints (Section 10) peaked at 12.5% greedy success and ended at 0.0%. This supports the interpretation that Wordle’s bottleneck for ES is not warm-start quality but the parameter-space estimator’s ability to extract useful credit from a sparse and noisy reward signal — the same signal REINFORCE was at least partially able to convert into a gradient through the action distribution.

8 Stage 4: Diagnosing the ES Failure

The repeated Wordle failures suggested that the main issue was not a single implementation bug or hyperparameter choice. Instead, the task seemed poorly matched to ES in this setting.

There were three main bottlenecks.

First, the reward was too sparse. Even with shaped rewards, most perturbations produced very similar outcomes. If nearly all perturbations receive the same reward, the ES update estimate has little useful directional information.

Second, the action space was too large. In the word-level setup, the policy had to choose among thousands of possible five-letter words. Small parameter perturbations usually did not create meaningful strategic changes; they often only changed the model’s output distribution in unstable or unhelpful ways.

Third, the trainable parameter space was still large relative to the reward signal. Even LoRA-based tuning involved millions of trainable parameters. ES gradient-estimate variance increases with the dimensionality of the search space, so weak reward differences become even harder to exploit.

We also analyzed the results through the lens of neural thickets theory. Neural thickets suggest that in sufficiently large pretrained models, good solutions may form dense regions in parameter space, making simple perturbation-based search more viable. However, our Gemma 1B Wordle experiments did not appear to be in such a regime. Random perturbation experiments found only sparse useful improvements, which helps explain why ES failed to produce reliable learning.

9 Stage 5: Pivot to Qwen and Character-Level Wordle

After the Gemma-based Wordle experiments failed, we changed both the model and the action formulation. We moved to Qwen-based Wordle models and changed the action space from word-level guessing to character-by-character generation.

The character-level formulation was intended to reduce the difficulty of the action space. Instead of selecting one word from thousands of possible guesses, the model emits one character at a time. This reduces the per-step action space to letters and better matches the autoregressive structure of language models. It also creates a more natural connection between Wordle feedback and individual character decisions.

The motivation for this pivot was that our earlier setup may have made exploration too hard. If ES could not find useful directions when entire words were treated as atomic actions, perhaps a character-level policy would provide a denser and smoother reward signal.

However, this change still did not lead to sustained ES improvement. The model remained difficult to optimize with parameter-space perturbations, suggesting that the problem was not only the size of the discrete action space. Even with a more language-model-native output format, the ES reward signal remained too noisy and brittle.

10 Stage 6: Prime Intellect Checkpoint Experiments

Because our own Wordle training did not produce a strong policy, we then investigated Prime Intellect’s released Wordle checkpoints. Their setup was stronger than ours in two important ways. First, they used an SFT warmup on multi-turn Wordle traces to teach the model the environment format and basic task behavior. Second, they applied RL against the Wordle environment, producing a released RL checkpoint that was substantially stronger than the SFT checkpoint.

This gave us a new experimental question. Instead of asking whether ES could discover a good Wordle policy from a weak initialization, we asked whether ES could improve a model that had already been placed near a successful Wordle policy. We tested ES from both Prime Intellect’s SFT checkpoint and their RL checkpoint.

Run	Init.	Stabilized?	Best eval	Best iter	Final eval	Final drift
SFT baseline	SFT	No	12.5%	8	0.0%	~ 9.42
RL baseline	RL	No	12.5%	1	0.0%	~ 0.29
RL + norm. ES	RL	Yes	12.5%	1	0.0%	~ 0.50

Table 2: Summary of the three Week 16 ES runs. All three runs reached the same best greedy evaluation success of 12.5%, corresponding to two wins out of 16 evaluation games, but none finished above 0.0% on the final checkpoint. The main difference was geometric: the SFT run drifted much farther from initialization, while the RL and normalized-RL runs stayed within a much smaller LoRA-parameter radius.

We ran three Week 16 ES jobs using Prime Intellect’s Qwen3-1.7B Wordle checkpoints. All three runs used LoRA with rank $r = 4$, thinking mode enabled, character-level action granularity, and 1,605,632 trainable parameters out of 1,722,180,608 total parameters. Each greedy evaluation used 16 Wordle games, so success rates are quantized in increments of $1/16 = 6.25\%$. Therefore, a reported 12.5% success rate corresponds to only two successful games.

The three runs differed in initialization and ES update stabilization:

- **SFT baseline:** initialized from PrimeIntellect/Qwen3-1.7B-Wordle-SFT.
- **RL baseline:** initialized from PrimeIntellect/Qwen3-1.7B-Wordle-RL.
- **RL + normalized ES:** initialized from the RL checkpoint, but with normalized ES updates enabled.

10.1 SFT Initialization

The first run initialized ES from PrimeIntellect/Qwen3-1.7B-Wordle-SFT. This checkpoint had already been trained to understand the Wordle environment format, making it a stronger starting point than our earlier Gemma-based warm starts.

However, ES did not produce sustained greedy Wordle performance from this initialization. Evaluation success peaked at 12.5% at iteration 8, corresponding to two wins out of 16 evaluation games, but the final checkpoint returned to 0.0% success. Most evaluations remained close to the six-turn cap, indicating that the trajectory did not produce a stable solving policy.

This run was also geometrically unstable. The initial calibrated raw ES gradient norm was 933.7, with an initial step target near 0.13, but later iterations showed much larger update scales. The applied gradient norm spiked to 52,566.27 at iteration 4, and the parameter drift $\|\theta - \theta_0\|$ grew to approximately 9.42 by iteration 14. Thus, the SFT-initialized run showed large, bursty ES updates without durable improvement.

10.2 RL Initialization

The second run initialized ES from PrimeIntellect/Qwen3-1.7B-Wordle-RL. This was the strongest initialization we tested, since the checkpoint had already undergone RL training in the Wordle environment.

Despite the stronger starting point, the evaluation result was essentially unchanged. Greedy success reached 12.5% at iterations 1 and 8, but the final checkpoint again evaluated at 0.0%. The saved best-so-far checkpoint was iteration 1.

The RL initialization did improve the geometry of training. Compared with the SFT run, the final parameter drift was much smaller, ending around

$$\|\theta - \theta_0\| \approx 0.29.$$

The run still showed bursty raw gradient norms, including values around 19,000 at several iterations, but these did not translate into the much larger drift seen in the SFT run. This suggests that the RL checkpoint produced a numerically healthier ES trajectory, but not a better task-level learning trajectory.

10.3 RL Initialization with Normalized ES Updates

The third run also initialized from `PrimeIntellect/Qwen3-1.7B-Wordle-RL`, but enabled normalized ES updates. This forced each ES update to have approximately fixed norm:

$$\|\Delta\theta\| \approx 0.13.$$

This run provided the cleanest test of whether uncontrolled update scale was the main problem. Normalization worked as intended: every iteration reported $\|\Delta\theta\| = 0.1300$, and cumulative parameter drift increased smoothly from 0.13 to approximately 0.50.

However, task-level performance still did not improve. Greedy evaluation success again peaked at 12.5% at iterations 1 and 8, but the final checkpoint evaluated at 0.0%. Raw gradient norms remained noisy and sometimes very large, including values above 18,000 and 53,000, but normalization prevented those spikes from causing uncontrolled parameter jumps.

This result separates update stability from task progress. Normalized ES made the parameter trajectory cleaner, but it did not create a useful Wordle learning signal.

11 Overall Results

Across the full project, the results followed a consistent pattern: ES worked in the small proof-of-concept setting, but did not produce sustained improvement once we moved to Wordle. That pattern held across every progressively stronger setup we tried, and the value of the project ended up being in the diagnostic detail of *how* each upgrade failed rather than in any single headline number.

In the Gridworld proof-of-concept (Section 4), ES reached 100% success on the base layout and adapted reasonably across perturbed layouts, confirming that the perturbation sampling, fitness evaluation, parameter-update loop, and curriculum machinery were all functioning. Once we moved to Gemma 3 1B IT Wordle (Section 5), the same template collapsed. On the 8-word mock vocabulary with a head-only policy, ES reached roughly 83% greedy success at iteration 9, but on the full $\sim 2,315$ -word answer pool with LoRA adapters every variant we tried stayed near 0% greedy success. The transition between the two regimes was a regime change, not a gradual degradation.

Section 6’s optimization push — stronger and weaker supervised warm starts, alternative curriculum schedules [16, 32, 64, 96, 128, 256, 512, 1024], LoRA ranks from $r=2$ to $r=32$, EMA momentum on the gradient estimate, antithetic $\pm\epsilon$ sampling, longer training horizons, and partial-feedback reward shaping — did not move the headline metric. Diagnostic logging showed that successive ES gradient estimates were essentially uncorrelated ($\cos(\hat{g}_t, \hat{g}_{t-1}) \approx 0$) and that the per-secret revisit count under common random numbers fell from one visit per iteration on the 16-word stage to roughly one visit per 60 iterations on the 1024-word stage, so the “winners” of an ES update were

largely those members who happened to draw easier secrets rather than those whose perturbation produced a better policy. This is the signal-density bottleneck identified in Section 8.

The REINFORCE baseline (Section 7) outperformed ES on the matched 32-word configuration on the same Gemma backbone. The cold REINFORCE run reached 41.7% greedy success and the warm-started REINFORCE run reached 16.7%; both are well above the corresponding ES runs, which sat near 0% on the full-vocabulary curriculum and peaked at only 12.5% even on the much stronger Week 16 Qwen initialization. We do not claim REINFORCE is strictly better than ES in general; what we do claim is that, on this task and this backbone, the policy-gradient estimator extracted information from sparse wins that the parameter-space ES estimator did not.

The pivot to Qwen3-1.7B with a character-level autoregressive policy (Section 9) was intended to make the per-step action space dense enough for ES to find signal, but it did not produce sustained improvement either. Finally, the Week 16 experiments (Section 10) loaded Prime Intellect’s published Wordle checkpoints (PrimeIntellect/Qwen3-1.7B-Wordle-SFT) and (PrimeIntellect/Qwen3-1.7B-Wordle-RL) directly as the ES starting point, skipping our own warm-start entirely. The SFT-initialized run, the RL-initialized run, and the RL-initialized run with normalized ES updates all peaked at 12.5% greedy success on the 16-game evaluation slate and ended at 0.0% on the final checkpoint (Table 2 and Figures ??–??).

The Prime Intellect runs are the most important ablation in the project because they rule out a simple alternative explanation for the earlier failures. ES did not fail on Wordle only because our warm-start was too weak — when we replaced our warm-start with a published checkpoint that is, by construction, already near a working Wordle policy, ES still could not turn that proximity into measurable improvement. Within the Week 16 runs themselves, the visible difference across configurations is optimization geometry rather than task performance: the SFT-initialized run drifts to $\|\theta - \theta_0\| \approx 9.42$ with bursty applied-gradient norms above 5×10^4 , the RL-initialized run stays inside a much smaller radius near 0.29, and the RL run with normalized ES updates keeps every step at $\|\Delta\theta\| \approx 0.13$ and grows smoothly to roughly 0.50. The training trajectories are visibly healthier as we move down that list, but the final Wordle success number does not change.

The single coherent reading of all of these results is that we made the optimization geometry progressively cleaner — warm start, curriculum, LoRA, character-level actions, stronger initialization, update normalization — without ever making the reward landscape itself dense enough for parameter-space search to climb it. That is what Section 12 takes up in more detail.

12 Discussion

The main lesson is that ES was not limited only by engineering instability. Some engineering changes did help. In particular, normalized ES updates clearly controlled the scale of parameter movement, and the RL checkpoint initialization produced much smaller drift than the SFT checkpoint initialization. However, these improvements affected optimization geometry more than task performance.

The bottleneck appears to be the reward signal. In Wordle, small changes to the policy often do not produce reliable differences in win rate. With only 16 greedy evaluation games, measured success is also extremely coarse: one additional win changes the reported success rate by 6.25 percentage points. As a result, isolated spikes to 12.5% success should not be interpreted as robust improvement.

The Prime Intellect experiments sharpen this conclusion. Their RL-trained checkpoint showed that Qwen can learn meaningful Wordle behavior under a stronger RL setup. However, our ES experiments could not further improve either their SFT or RL checkpoints. This suggests that the

issue was not simply model capacity or lack of Wordle knowledge. Instead, ES could not extract a reliable enough parameter-space learning signal from the environment under our setup.

This also helps explain the gap between Gridworld and Wordle. Gridworld has a small state space, simple dynamics, and a low-dimensional policy. Wordle with a pretrained language model has a much larger parameter space, a multi-turn discrete action structure, and a sparse success condition. Even with LoRA restricting the number of trainable parameters, the search space remained too large relative to the available reward signal.

13 Limitations and Caveats

The negative result reported in this paper deserves several caveats documented in our weekly self-critiques in `docs/critiques/`. Reading them as outright "ES cannot solve Wordle" overclaims; the more careful reading is "ES at the scale and budget we ran did not measurably improve any of the Wordle policies we tried." Five caveats matter most.

Population size and iteration budget were small for ES. The Week 16 Prime Intellect runs used $N_{\text{pop}} = 8$ and $N_{\text{iter}} = 15$. Salimans et al. (2017) ran ES with populations in the thousands; our internal Week 12 Session 8 PASS-A result (which did move the metric meaningfully at `VOCAB=16`) needed $N_{\text{iter}} = 60$ and reached $\|\theta - \theta_0\| \approx 4.9$. The Week 16 trajectories reached at most $\|\theta - \theta_0\| \approx 0.50$, roughly one-tenth of that displacement. Each Week 16 ES iteration on Qwen3-1.7B with `enable_thinking=True` and `MAX_NEW_TOKENS=512` requires $(2N_{\text{pop}} + 1) \cdot N_{\text{eval}} \cdot 6$ `model.generate` calls, so larger populations were infeasible on our compute budget; but the honest read of three flat runs at $N_{\text{pop}} = 8$ is that we did not give ES enough population to estimate a credible gradient on a 1,605,632-dimensional LoRA surface, not that ES could not work here in principle.

Evaluation slates are small and quantized. Greedy evaluation in Week 16 used 16 games per checkpoint, so success rates are quantized in 6.25 percentage-point increments. A peak of 12.5% corresponds to exactly two wins. Under a Bernoulli null at either checkpoint's measured cold-base rate ($\text{SFT}_{\text{cold}} \approx 6\%$, $\text{RL}_{\text{cold}} \approx 12\%$), two-out-of-sixteen is well inside the 95% confidence interval. The "peak" should therefore be read as "did not falsify the null," not as evidence of meaningful improvement.

The Week 16 parser dominated the Week 16 policy. Across all three Week 16 runs the verbose log reported a `fb% = 100%` `<guess>`-parse-failure rate, meaning every guess was selected by the layered fallback in `src/wordle_qwen_policy.py:parse_word_from_text` (strip `<think>` blocks, take the first 5-letter alphabetic run that is not in a hard-coded denylist, otherwise return the `XXXXX` sentinel) rather than by the model's `<guess>` tag. ES cannot improve a policy whose guess is determined by string post-processing rather than by the model distribution. This caveat does not change our final conclusion, but it does mean Week 16's "ES did not improve the RL checkpoint" is at least partly "the parser was doing the policy's job."

Single-seed runs throughout. The Week 16 SFT-init, RL-init, and RL-init + normalized-ES runs were all performed with a single random seed (`SEED = 42`). Given the per-iteration eval noise floor at one episode per secret on a 16-secret slate ($\sigma \approx 8$ percentage points), a single-seed *best* = 12.5% is uncalibrated as an effect size. Multi-seed runs would have been the cheapest possible falsification of the "two-wins-is-a-peak" reading and were not done.

Cold-base and ES eval were not RNG-aligned. The cold-base evaluations of Qwen3-1.7B-Wordle-SFT and -RL and the post-ES evaluations used independent RNG streams. This is why the FINAL AT-TRIBUTION block in each console log prints `ES contribution: n/a`: we cannot subtract the cold baseline from the ES result and obtain a meaningful Δ . The qualitative conclusion ("ES did not visibly help") still holds because both numbers are inside the same quantization bucket, but the quantitative attribution does not.

14 Conclusion

We investigated Evolution Strategies as a gradient-free optimization method for sparse-reward reinforcement learning tasks. ES succeeded on a small Gridworld proof-of-concept, confirming that the implementation worked in a simple sparse-reward environment. However, ES failed to scale to Wordle, even after extensive attempts to improve the setup.

Our experiments progressed from Gemma-based word-level Wordle policies, to supervised warm starts and curriculum learning, to Qwen-based character-level policies, and finally to ES tuning from Prime Intellect’s released SFT and RL Wordle checkpoints. None of these stages produced sustained Wordle improvement. The strongest final result was a set of isolated 12.5% success spikes in 16-game evaluations, but all three final Prime Intellect checkpoint runs ended at 0.0% final greedy success.

The final conclusion is negative but useful: ES can be attractive because of its simplicity and communication efficiency, but for Wordle-scale language-agent tasks, the reward landscape was too sparse and brittle for ES to make reliable progress. Stronger warm starts and better update stabilization improved the training geometry, but they did not solve the central credit-assignment problem.